**OBJECT ORIENTED PROGRAMMING**
**CCS20704, DCS20504, CCS23803**

---

**Received Date : 3 September 2026**
**Submission Due Date: 23 October 2026**
**Lecturer : Dr. Jamal Abdullahi**
**Weightage : 15%**
**Semester : SEPTEMBER 2026**

---

**Instruction to students:**

- This is a GROUP assignment.
- Complete this cover sheet and attach it to your assignment (first page).

| **Student declaration:** | |
|---|---|
| ***I declare that:***<br>• ***This assignment is my/our own work.***<br>• ***I/we understand what is meant by plagiarism.***<br>• ***My lecturer has the right to deduct my marks in case of:***<br>- ***Late submission***<br>- ***Any plagiarism found in my assignment.*** | |
| **NAME: Felisa Chin Sze Min**<br>**MATRICS NO: 012026021292**<br>  | **NAME: Nurul Jannah Binti Mohd Azrin**<br>  <br>**MATRICS NO: 012026070294** |
| | |

## 1.0 INTRODUCTION

Object-Oriented Programming (OOP) is a technique for organizing code in classes and objects. A class defines the nature of an object by describing its attributes and methods. An object is an instance of a class. Different classes can be defined as per the nature of the system, and objects can be created of the class. Objects can be organized in groups as per the nature of relationship between the objects. This facilitates in modularizing and organizing code efficiently.

For this assignment, we will develop a Student Management System in Java. The system will have a Student class to store student information, i.e., student ID, name, age, course, semester, CGPA and registration status. The class will have constructors, getter and setter methods and other methods to manage student information.

The program incorporates important OOP concepts, especially encapsulation, using private attributes and getter and setter methods. The program uses selection and looping structures in the main class to check registration status, show academic performance and explain semester progress. Overall, the assignment intends to incorporate fundamental concepts of OOP in the context of a real-world, working Java program.

## 2.0 OBJECTIVES

The objectives of this assignment are:

1. To develop and implement a Student class with at least seven attributes and appropriate methods.
2. To apply encapsulation using private attributes, constructors, getters, and setters.
3. To use selection and looping structures in Java while doing student registration checks and semester progress checks.
4. To create a functional Java application for displaying, adding and/or updating student records.

## 3.0 CLASS DESIGN

The Student class represents a student in the Student Management System. It contains seven private attributes to store student's personal and academic information. The class contains a couple of constructors, setter and getter methods and other methods that are concerned with management of student information.

Table 1: Student Class Attributes

| Attribute | Data Type | Description |
|---|---|---|
| studentId | String | Stores the student's identification or matric number |
| name | String | Stores the student's name |
| age | int | Stores the student's age |
| course | String | Stores the student's course |
| semester | int | Stores the current semester |
| cgpa | double | Stores the student's Cumulative Grade Point Average |
| isRegistered | boolean | Indicates whether the student is registered |

The class implements encapsulation by declaring all attributes as private. The attributes have getter and setter methods. The setter methods include validation to avoid the setting of illegal age, semester, and CGPA. The class provides methods to register students, display information, check the status, and progress to the next semester.

## 4.0 IMPLEMENTATION

The Student Management System was implemented using Java and consists of the Student class and the Main class. The Student class stores student information and provides methods to manage and display the data.

### 4.1 Constructors

The class has two constructors. The default constructor sets the attributes to predefined values, such as "Unknown" for text attributes, 0 for age, 1 for semester, 0.0 for CGPA and false for registration status. The parameterized constructor is used when a student object is created and setter methods are used to set the values.

### 4.2 Getters and Setters

Seven getter methods allow the user to retrieve the values of the student attributes. The attributes are modified through the use of 7 setter methods. The attributes are declared as private to support encapsulation. Basic validation is done in the setters for age, semester, and CGPA to make sure the entered values are within the valid range.

### 4.3 Additional Methods

The class includes five additional methods. The registerStudent() method changes a student's registration status to true, and displayStudentInfo() method shows the student's information and academic status. The checkAcademicStatus() method determines student's academic status based on their CGPA. The checkRegistration() method determines if student is registered, and the updateSemester() method advances the student to the next semester, only if the total number of semesters is less than or equal to 20.

Source Code Repository

The complete Java source code for the Student Management System, including the Student and Main classes, is available on GitHub at:

https://github.com/smfelisa/OBJECT-ORIENTED-PROGRAMMING-Assignment-1.git

## 5.0 PROGRAM OUTPUT & TESTING

The program showcases the features of Student Management System. It illustrates creating objects of Student class using different constructors and displaying the information created. The program illustrates the use of getter and setter methods. It demonstrates checking the student's academic performance and registration status, shows the student's semester progress and updates the student's semester.

The following test cases can be used to verify the program's functionality.

Table 2: Program Testing

| Test Case | Expected Result |
|---|---|
| Create student object | Student information is initialized using both constructors. |
| Display student information | All seven attributes and academic status are displayed. |
| Update CGPA | The CGPA changes to the new valid value. |
| Check registration | The program identifies whether a student is registered and registers the first student if necessary. |
| Check academic status | An appropriate message is displayed based on the student's CGPA. |
| Display semester progress | The program displays semesters from 1 to the student's current semester. |
| Update semester | The student's semester increases by one, provided it does not exceed 20. |

The program was run to test the Student Management System. The output in Figure 1 shows both student objects, their attributes, and the use of getter methods to update the CGPA of the first student. Figure 2 illustrates the registration check and selection of academic performance. The figure also shows the progress of the semester using a loop. Figure 3 shows the program completion message and the review of both students' registrations for the next semester. The output shows the student management system performs the intended functions.

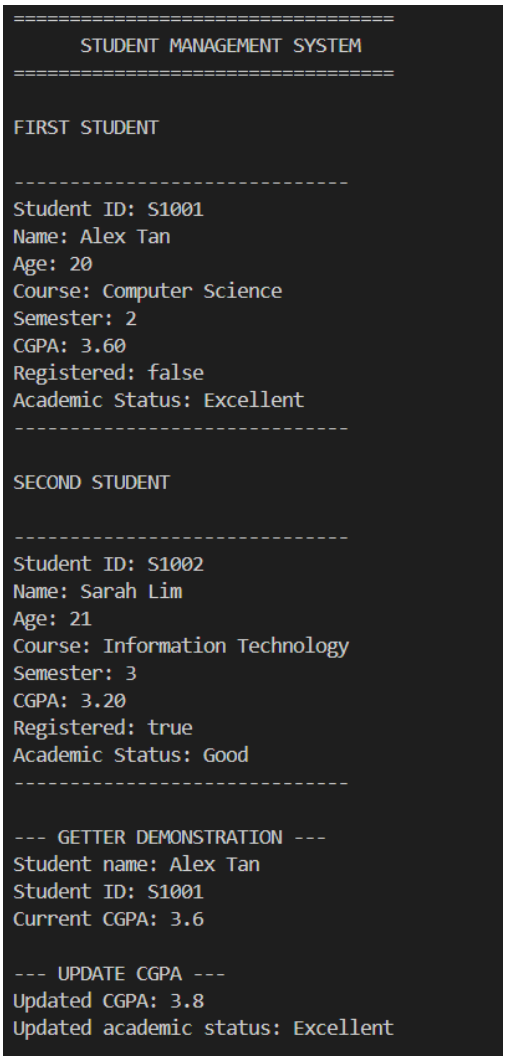



Figure 1: Output Showing Student Information, Getter Methods, and CGPA Update

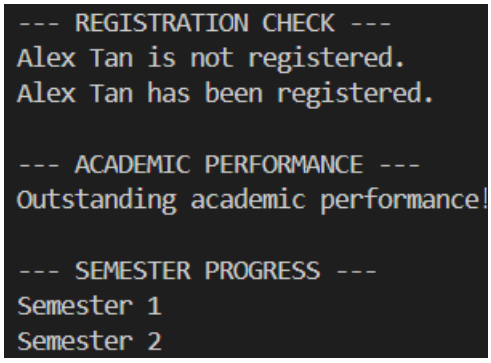



Figure 2: Output Showing Registration Check, Academic Performance, and Semester Progress

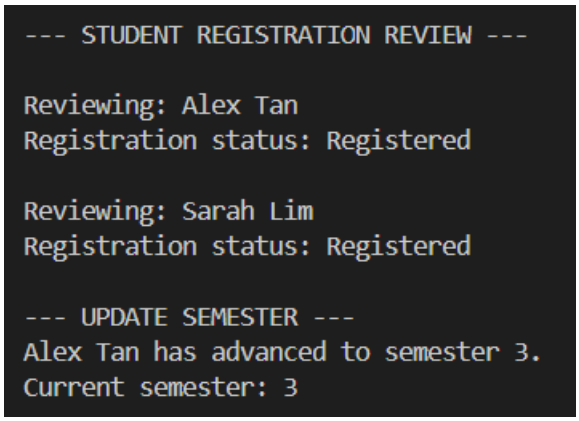



Figure 3: Output Showing Student Registration Review and Semester Update

## 6.0 CONCLUSION

In summary, this assignment provided practical experience developing a Java program using Object-Oriented Programming (OOP). The Student Management System illustrates use of classes and objects, constructors, encapsulation, getters and setters, methods and other related operations to manipulate student information. The program also uses control structures, like selection and looping, to verify registration status, and to present and analyze student's academic performance and progress, respectively. This assignment enhanced our knowledge in Java Programming and practical use of Object Oriented Programming.

**REFERENCES**

Oracle. (n.d.). *Classes and objects*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/javaOO/classes.html

Oracle. (n.d.). *Controlling access to members of a class*. The Java Tutorials. https://docs.oracle.com/javase/tutorial/java/javaOO/accesscontrol.html